Phases 2–5 Summary (CRAIG reproduction)

Every number below comes from `results/raw/*.csv` (aggregated by `analysis/runtime_analysis.py` into `results/processed/`). Hardware: Linux box, CPU only (8 cores), PyTorch CPU build. Tests: `pytest tests` → **23 passed, 0 failed**.

What was implemented

- `craig/gradients.py`: per-sample grads [N, D] using `vmap(grad(...))` over `functional_call`, with chunking. If vmap fails, it falls back to one backward pass per sample. Gradients are computed in eval mode, so BatchNorm uses running stats.
- `craig/representations.py`: full gradient / last layer (classifier head) / JL random projection (Gaussian or Rademacher).
- `craig/similarity.py`: blockwise Euclidean and cosine distances with a 4 GiB memory guard; s = max(D) − d.
- `craig/facility_location.py`: naive greedy and lazy greedy (Minoux) facility location.
- `craig/weights.py`: Voronoi weights w_j = |C_j|, with Σw = N.
- `craig/selector.py`: `CraigSelector` selects per class and times each stage separately.
- `analysis/gradient_error.py`, `analysis/runtime_analysis.py`, `analysis/plots.py`.
- `experiments/common.py`, `run_mnist.py`, `run_cifar10.py`, `run_timeseries.py`, `run_scaling.py`.

Protocol notes: - Selection runs **once, at the seed's initial weights**. There is no reselection each epoch, unlike the paper's deep-net setup. - CRAIG time = gradients + similarity + facility location + weights + training. It is recomputed for every (seed, ratio). - `training_time` does not include test evaluation. - Gradient error is measured at the initial weights, against Σ_i g_i. RANDOM uses weights N/k.

MNIST: logistic regression, full 60k train / 10k test, 20 epochs, SGD lr 0.01, 5 seeds (`mnist_2026_10_29_001.csv`)

ratio	CRAIG acc	CRAIG-unweighted	RANDOM acc	CRAIG selection s	CRAIG total s	rel. grad err CRAIG / RANDOM
5%	90.19 ± 0.15	89.53	89.68 ± 0.18	29.4	29.9	0.118 / 0.128
10%	91.07 ± 0.11	89.81	90.45 ± 0.21	31.8	32.7	0.087 / 0.090
20%	91.59 ± 0.16	89.98	91.15 ± 0.34	34.0	35.7	0.061 / 0.063
30%	91.71 ± 0.19	90.59	91.51 ± 0.29	35.4	38.0	0.047 / 0.048
50%	92.14 ± 0.18	91.57	91.89 ± 0.14	38.2	42.4	0.027 / 0.030

FULL: 92.22 ± 0.12 %, training time 6.59 s.

- Weighted CRAIG beats RANDOM at every ratio, by 0.2–0.6 points.
- Without weights, CRAIG is *worse* than RANDOM, so the weights matter.
- **Break-even fails on MNIST.** Selection (~29–38 s, mostly lazy greedy at 23–29 s) is 4.5–6× longer than full training (6.6 s). End-to-end speedup is 0.16–0.22×; training-only speedup is 1.6–14.3×.

CIFAR-10: REDUCED scale (`cifar10_2026_10_29_001.csv`)

- Setup: first 10,000 training images, full 10k test set, CIFARCNN, 10 epochs, 3 seeds.
- CRAIG uses last-layer gradients.
- Accuracy (%): FULL 59.02.

ratio	CRAIG	CRAIG-unweighted	RANDOM
10%	37.00	35.00	36.13
20%	43.19	43.38	44.01
30%	45.16	47.46	47.55
50%	50.14	51.52	51.68

- CRAIG beats RANDOM only at 10%. With one-shot selection at initialisation, it does not help this CNN.
- Its relative gradient error is still far lower than RANDOM's at small ratios (0.21 vs 0.80 at 10%).
- Selection takes about 1.0 s, so CRAIG breaks even at every ratio (end-to-end speedup 1.75–5.07×). But accuracy is 9–22 points below FULL.

Time series: synthetic sinusoids, 1D CNN, N=4000, L=100, 4 classes, 10 epochs Adam, 3 seeds

No real UCR data was bundled.

- **Easy setting** (noise std 0.2, `timeseries_2026_10_29_001.csv`): all methods reach 100% at every ratio, so it cannot tell them apart.
- **Hard setting** (noise std 2.0, `timeseries_2026_10_29_002.csv`): results are unstable. FULL scores 52.9 ± 5.9 %, *below* several subsets, and some cells have std up to 16 points.
- CRAIG / RANDOM accuracy (%): 5%: 47.3 / 48.1; 10%: 54.0 / 58.8; 20%: 64.0 / 62.8; 30%: 50.7 / 55.9; 50%: 54.8 / 61.9.
- **No reliable conclusion can be drawn on accuracy.** The training setup needs tuning; one likely cause is train/eval BatchNorm instability.
- CRAIG's gradient error is much lower than RANDOM's: 0.07 vs 0.66 at 10%.
- Selection takes about 0.7 s, so CRAIG breaks even at every ratio (1.38–2.92×).

Scaling (2 seeds, ratio 10%, 10 epochs)

- **Dataset size** (MNIST LR, `scaling_size_*.csv`). Selection time grows superlinearly with N, while FULL training grows linearly:

N	CRAIG selection s	FULL training s	CRAIG end-to-end speedup	CRAIG acc	RANDOM acc
1k	0.026	0.050	1.50×	76.7%	72.8%
5k	0.19	0.25	1.12×	86.7%	85.1%
10k	0.58	0.49	0.75×	88.7%	87.8%
20k	3.01	1.03	0.33×	89.5%	89.0%

The break-even point lies between N = 5k and 10k in this setup. - **Sequence length** (synthetic time series, N = 2000, `scaling_seq_*.csv`): - Gradient time rises from 0.13 s at L = 50 to 1.50 s at L = 1000. Similarity and facility-location time stay flat, because D does not depend on L. - CRAIG breaks even at

every L (2.25–3.0×). - Accuracy: CRAIG is *below* RANDOM for L ≥ 200 (e.g. L = 1000: 54.6% vs 64.3%), with large variance.

Figures (`results/figures/`)

- Per run: `<csv-stem>_accuracy_vs_ratio.png`, `_craig_time_breakdown.png`, `_speedup_breakeven.png`, `_gradient_error.png`, `_accuracy_vs_total_time.png`. These exist for mnist_2026_10_29_001, cifar10_2026_10_29_001, timeseries_2026_10_29_001 and timeseries_2026_10_29_002.
- Scaling: `scaling_dataset_size.png`, `scaling_sequence_length.png`.

Not done / limitations

- CIFAR-10 was run only at reduced scale: 10k of 50k images, 10 epochs, 3 seeds.
- There is no per-epoch reselection for deep nets, and no stochastic-greedy speedup. The lazy greedy written in Python is the main bottleneck on MNIST.
- No real time-series dataset was used (FordA etc. are not downloaded).
- MPS and CUDA code paths exist (device auto-detection and sync) but were not tested; there is no GPU here.
- The original Phase-1 CSV (`mnist_2026_09_23_001`) used 1000 samples and 2 epochs. It is not comparable to the runs above.